

From Browser to Pico: Helping Students Build Their First Neural Network

Neural networks have become part of many school curricula, yet they remain surprisingly difficult to teach. Modern AI tools can produce impressive predictions with only a few clicks, but they rarely reveal what happens between entering data and receiving a result. For many learners, the neural network itself remains a black box.

AI-ANNE (A Neural Net for Exploration) was created to make that invisible learning process visible.

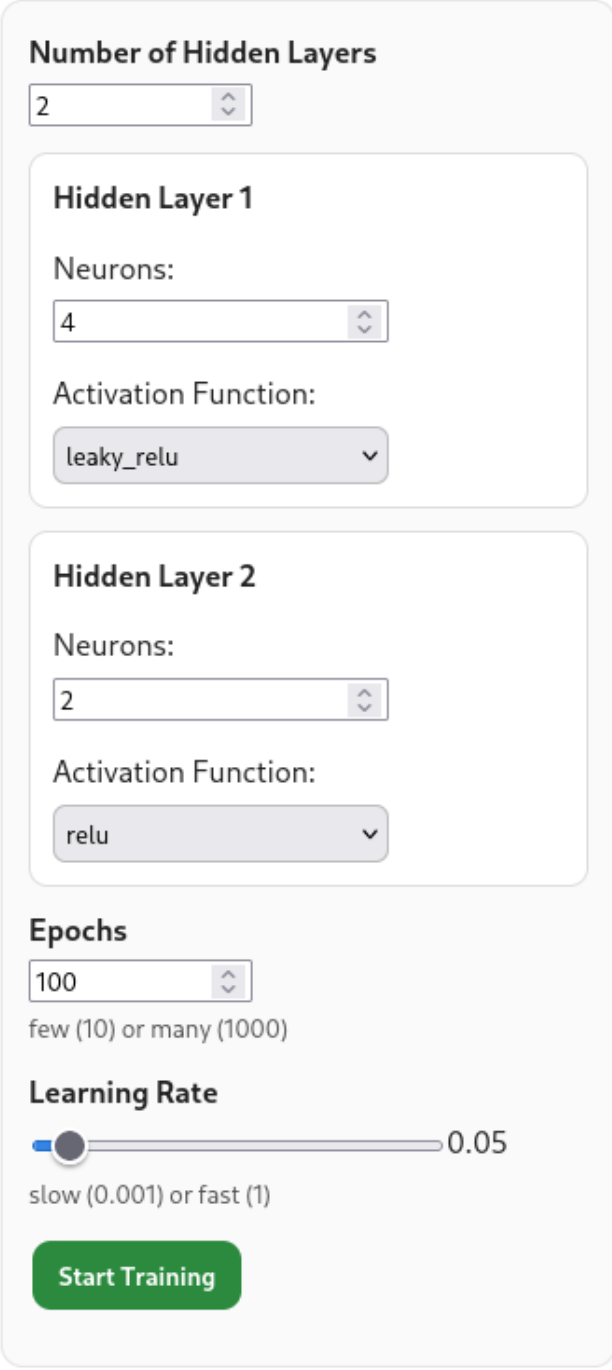
Instead of focusing on building ever larger models, AI-ANNE encourages students to explore how a neural network actually learns. Every important part of the process remains accessible - from the training data and network architecture to the learned weights, biases, activation functions and the final prediction. Rather than simply producing an answer, the software encourages learners to ask questions such as *What happens if I add another hidden layer?* or *How does changing the learning rate affect the model?*

Choosing a neural network

Getting started requires nothing more than a web browser. Students can upload their own CSV dataset or begin with one of two included demonstration datasets. A standard mode provides predefined network architectures for beginners, while an advanced mode allows learners to specify the number of hidden layers, neurons and activation functions.

This flexibility makes AI-ANNE suitable for different levels of experience. Beginners can focus on understanding the basic concepts of supervised learning, while more advanced students can investigate how architectural decisions influence model performance. Because the complete

workflow runs inside the browser, no software installation or specialist machine-learning libraries are required.



The interface is a light gray panel with rounded corners. At the top, it has a section titled "Number of Hidden Layers" with a dropdown menu set to "2". Below this are two sections for hidden layers. "Hidden Layer 1" has a "Neurons:" dropdown set to "4" and an "Activation Function:" dropdown set to "leaky_relu". "Hidden Layer 2" has a "Neurons:" dropdown set to "2" and an "Activation Function:" dropdown set to "relu". Below these is an "Epochs" section with a dropdown set to "100" and text "few (10) or many (1000)". The "Learning Rate" section features a slider with a blue circle at the left end, labeled "0.05" at the right end, and text "slow (0.001) or fast (1)" below it. At the bottom is a green "Start Training" button.

Figure 1: Selecting a neural network architecture

Watching a network learn

Once training begins, AI-ANNE visualises the neural network while simultaneously exposing the underlying hyperparameters. Students can observe how the selected architecture learns from the data and compare different configurations simply by retraining the model.

After training, learners can inspect the confusion matrix, prediction accuracy and - most importantly - the learned weights and biases that define the neural network. These parameters are no longer abstract mathematical symbols found only in textbooks; they become concrete values that students can compare after changing the architecture or training settings. Small modifications to the learning rate or network structure immediately become visible in the resulting model, encouraging experimentation rather than memorisation.

Understanding a prediction

Learning does not stop once the model has been trained. Students can enter their own feature values and immediately test the network with new examples. AI-ANNE highlights the activation pathway that produced the prediction and indicates which input feature had the strongest influence on the result. Instead of treating the prediction as an unquestionable answer, learners are encouraged to investigate *why* the network reached its decision and how changing individual inputs affects the outcome.

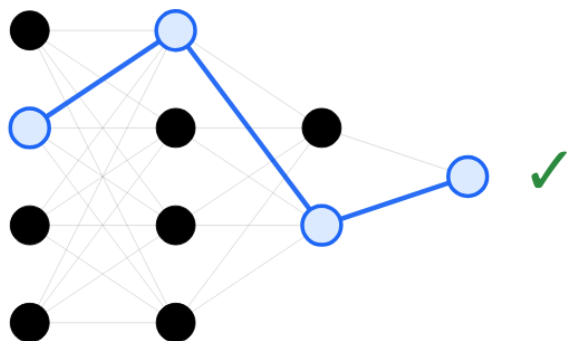


Figure 2: Exploring a prediction and visualising the activation pathway through the network.

This often leads to valuable classroom discussions. *Why does changing one feature alter the prediction? Why does adding another hidden layer improve one dataset but not another? Which architecture is sufficiently complex - and when does a model become unnecessarily complicated?* By experimenting with their own networks, students discover that designing an AI model

always involves making informed decisions rather than simply pressing a button.

From browser to Raspberry Pi Pico

Trained Parameters and Architecture

Hidden Layer 1:

```
w1 = [[-0.20963, 2.11224, -1.64772, -1.06458],
      [-1.11364, -0.89338, 3.76956, 2.52785],
      [0.25186, -0.03789, -0.49455, 0.20523],
      [-2.09792, -1.45329, 6.64842, 4.03302]]
b1 = [0.19337, -1.35732, -0.04618, -2.24280]
```

Hidden Layer 2:

```
w2 = [[0.30941, 0.18229, 0.10810, -0.30348],
      [-1.46288, 1.61086, -0.46697, 2.80096]]
b2 = [-0.28561, -0.48290]
```

Output Layer:

```
w3 = [[0.24323, 2.88970]]
b3 = [-5.08676]
```

Activation Functions:

Hidden Layer 1: leaky_relu

Hidden Layer 2: relu

Output Layer: sigmoid

Architecture: [4, 4, 2, 1]

Figure 3: Exporting the trained model and running it on a Raspberry Pi Pico

Perhaps the most rewarding step comes at the end of the activity.

With a single click, the trained neural network can be exported as MicroPython code for a Raspberry Pi Pico. The same architecture, learned weights, biases and activation functions that were explored in the browser are transferred directly onto the microcontroller. Students therefore experience the complete journey - from training a model on their computer to deploying exactly the same neural network on physical hardware.

This connection between browser-based experimentation and physical computing creates a learning experience that is difficult to achieve with conventional machine-learning frameworks. Students are not simply using artificial intelligence - they are building it, analysing it and finally deploying it themselves. Because every design decision remains visible throughout the process, discussions naturally emerge about model complexity, explainability, overfitting and the

trade-offs between prediction quality and computational resources.

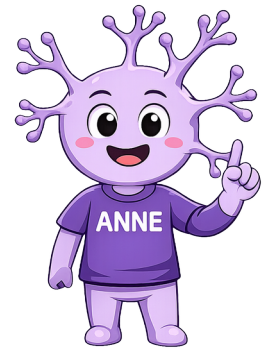
Perhaps the most valuable lesson is that students leave the activity with more than a trained model. They understand that every prediction is the result of many simple mathematical operations that are transparent, inspectable and portable. By the end of the lesson, the neural network is no longer a mysterious *black box* - it is their own creation, running on a Raspberry Pi Pico.

[Export Parameters](#)[MicroPython Code](#)

Figure 4: Easy parameter and model export

Getting started

AI-ANNE is released as free and open-source software to support AI education in schools, vocational schools, universities and self-directed learning. The browser application, source code and further documentation are freely available at:



More information about AI-ANNE in The Journal of Open Source Software (JOSS)





AI-ANNE: (A) (N)eural (N)et for (E)xploration

Dennis Klinkhammer ¹

1 Statistical Thinking (Germany), www.statistical-thinking.de, info@statistical-thinking.de

DOI: [10.21105/joss.08039](https://doi.org/10.21105/joss.08039)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Øystein Sørensen](#) 

Reviewers:

- [@samiralavi](#)
- [@kalpan80](#)

Submitted: 16 March 2025
Published: 09 June 2025

License:
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Machine learning and deep learning are increasingly driving innovation across various fields (LeCun et al., 2015) and can also be transferred on microcontrollers (Ray, 2022), for example to process sensor data (Cioffi et al., 2020). Since microcontrollers have limited computational resources (Delnevo et al., 2023), training or developing neural networks on microcontrollers remains a complex challenge (Wulfert et al., 2024). However, MicroPython as programming language enables a resource-efficient implementation of neural networks on microcontrollers and provides insights into the fundamentals of neural networks in terms of explainable artificial intelligence (Haque et al., 2023; Meske et al., 2022), while coding from scratch in MicroPython also allows for a streamlined and tailored approach (Delnevo et al., 2023). AI-ANNE: (A) (N)eural (N)et for (E)xploration provides a framework that transfers neural networks from Python to MicroPython and enables the application of pre-trained TensorFlow and Keras models on microcontrollers as well as training directly on the microcontroller via forward and backward propagation. Furthermore, it enables users to explore the performance of neural networks while simultaneously the number of neurons and layers as well as the underlying activation functions can be adjusted easily in MicroPython, which makes it also suitable for didactic application (Collier & Powell, 2024; Meske et al., 2022; Scherer, 2016; Verma et al., 2022).

Statement of need

A previous investigation of skill requirements in artificial intelligence and machine learning job advertisements (Verma et al., 2022) served as a guide for the development of AI-ANNE: (A) (N)eural (N)et for (E)xploration. Therefore, it demystifies the mathematical principles underlying neural networks, allowing users to better understand the relationships between data, weights, biases and outputs (Frank et al., 2020; Scherer, 2016; Schmidt et al., 2020). This includes reducing the number of layers, optimizing the activation functions, or using specialized operations to reduce computational load (Sakr et al., 2021). Moreover, by developing the neural network directly in MicroPython, the model becomes highly portable and adaptable to various microcontroller architectures, such as the Raspberry Pi Pico or the Raspberry Pi Pico 2. Especially the step-by-step visibility in MicroPython helps users to identify and understand common challenges like vanishing or exploding gradients, while fostering a more intuitive grasp of how neural networks learn (Kong et al., 2022). Debugging the code in MicroPython further reinforces this knowledge and provides insights into how errors propagate and are corrected during training. This is in accordance with the primary goals of explainable artificial intelligence (Haque et al., 2023; Schmidt et al., 2020) and provides insights into artificial intelligence related responsibilities (Collier & Powell, 2024; Frank et al., 2020) as well as practical experiences (Li et al., 2021). As such, implementing neural networks on microcontrollers with AI-ANNE: (A) (N)eural (N)et for (E)xploration shall promote necessary innovation and problem-solving skills (Verma et al., 2022).

Klinkhammer, (2025). AI-ANNE: (A) (N)eural (N)et for (E)xploration. *Journal of Open Source Software*, 10(110), 8039. <https://doi.org/10.21105/joss.08039>



AI-ANNE EDUCATION (MANUAL)

(1) Objectives of AI-ANNE

Many AI tools help students understand how AI can be used. AI-ANNE shows how AI works. **Students interactively train neural networks**, experience the **importance of data quality**, and can **transparently examine** potential errors during training or when using **AI**. This allows them to improve their AI step by step and ultimately **create their own AI**. To achieve this, AI-ANNE enables students to **classify and distinguish between objects**, recognize objects using a camera (**Computer Vision**), or **train small language models with a real Generative Transformer**. All of this is fully **privacy-compliant**, requires no specialized hardware, and assumes no prior knowledge on the part of the students. Simply get started and try out your own AI.

(2) Didactic Principles

Many AI tools are black boxes. AI-ANNE is a glass box.

Students **train neural networks themselves** instead of simply chatting with AI or writing prompts. In doing so, they experience **the entire process** that makes AI possible: collecting and labelling data as well as training and testing models. With AI-ANNE, students can understand **why AI can make mistakes**, why **data quality is crucial**, and that AI tools are generally created and shaped by people.

AI Can't Know Everything!

AI-ANNE shows that AI only knows **what you show it**. Poor-quality data leads to poor AI responses. In this way, students develop **statistical thinking and learn to engage critically with AI**.

AI-ANNE Is Based on Learning by Doing.

With AI-ANNE, students can **interactively and easily** teach an AI to distinguish between everyday objects, let a neural network see geometric shapes through a camera, or even **train their own small language models**. This makes neurons, activation paths, and even the inner workings of a Generative Transformer transparent and easy to explore. **AI is not just explained – it is experienced.**

Open and Adaptable Solutions Foster Creativity and Allow for Target-Group-Specific Learning.

AI-ANNE is **open source**, freely customizable under the **MIT License**, and will **always remain free of charge**. Students and teachers can **contribute their own ideas**, develop new examples and applications, and work **entirely locally, privacy-compliantly, and independently**. And because AI-ANNE requires neither a cloud connection nor expensive hardware, this educational introduction to the world of AI training is also particularly **resource-efficient**.

(3) Brief Description of AI-ANNE

For students, AI-ANNE represents a **change in perspective**: away from simply using digital tools and towards a **deeper understanding of their underlying principles**. In this way, learners are not only prepared for a world of life and work increasingly shaped by AI, but are also empowered to become informed and self-determined individuals.



Classification for Distinguishing Between Objects

(0) Select Objects

First, you decide which two objects AI-ANNE should learn to distinguish between. You can upload **your own images of the objects** into the two designated fields for **Object (0)** and **Object (1)**, or choose one of the **predefined examples**. The example uses nuts and shows that chestnuts labelled as (0) represent a different object from acorns labelled as (1).

Beispiel (Noppensteine)

Beispiel (Nudelgerichte)

Beispiel (Nussfrüchte)

Objekt (0)



Bild hier ablegen oder klicken

Objekt (1)



Bild hier ablegen oder klicken

Bilder zurücksetzen

(1) Data

Next, the objects are described using **selected characteristics**, known as **features**. These features can describe the **physical characteristics of the objects** (e.g., 0.0 for angular and 1.0 for round), their **dimensions** (e.g., 4.5 cm in height and 4.0 cm in width), or their **colour** (e.g., 0.0 for dark and 1.0 for light).

Eigene Daten

Tipp: Trage hier zeilenweise die vier Features des jeweils gezeigten Objekts ein.

	1. Feature	2. Feature	3. Feature	4. Feature
	F1	F2	F3	F4
	F1	F2	F3	F4
	F1	F2	F3	F4
	F1	F2	F3	F4

Daten übernehmen

Tabelle leeren

It is important that the selected features are **always entered as numbers** and that a **decimal point is used as the decimal separator**. This is the standard convention in international programming.



(2) Neural Network

Select a Model

In the basic settings, you can choose between a **small neural network** and a **larger neural network** as the model. If four characteristics, known as **features**, are provided to the neural network, four so-called **neurons** are located on the left in the **input layer**. These features are processed through the neurons in the subsequent layers, known as **hidden layers**, until they reach the **output layer**. The single neuron in the output layer then makes a decision, for example between Object (0) and Object (1). Each neuron's decision is based on **mathematical calculations**.

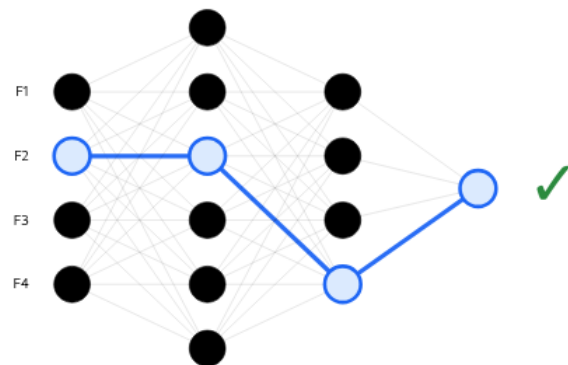
Modell auswählen

Epochen

wenige (10) oder viele (500)

Lernrate

langsam (0.001) oder schnell (1)



Epochs

In a neural network, an **epoch** means that the neural network **goes through all the training examples** once. It then looks at which tasks it performed well and where it still made mistakes, and tries to do better the next time. Multiple epochs therefore help the neural network **learn from its mistakes and become increasingly accurate**.

Learning Rate

The **learning rate** determines **how large the steps** a neural network takes **while learning** should be. When the neural network detects errors, it can make **only small corrections** or make **much larger adjustments**. The learning rate therefore determines whether the neural network learns from its mistakes **slowly in small steps** or more **quickly in larger steps**. A good learning rate helps the neural network improve **steadily, step by step**.

Activation Path

Later, once the neural network has been trained and an **object is to be classified**, the neural network also displays the so-called activation path. This illustrates how the objects' features contributed to the **neural network's decision-making** process and **which feature had the greatest influence on the decision**.



(3) Evaluation

The **confusion matrix** provides a clear overview of how accurately a neural network, used as a classification model, makes its predictions. It compares the **actual object classes** with the **predicted object classes** and checks, for example, whether an object predicted to be a chestnut (0) is actually a chestnut or an acorn (1). If, for example, two chestnuts (0) were used to train the neural network, there should also be two **True Negatives (TN)** with the value (0). For the acorns (1), these would correspondingly be **True Positives (TP)** with the value (1).

Konfusionsmatrix:

TN: 2 FP: 0

FN: 0 TP: 2

Genauigkeit:

1.00

🎉?? Super! Das Netzwerk hat gut gelernt!

If the neural network classifies an acorn (1) as a chestnut (0), this results in a **False Negative (FN)**. If the neural network classifies a chestnut (0) as an acorn (1), this results in a **False Positive (FP)**. If a neural network produces too many False Negatives and False Positives, its **accuracy** decreases.

(4) Prediction

In the final step, the trained neural network can be used for the **classification of objects**. New chestnuts (0) and acorns (1) that were not used for training can now be recognized.

1. Feature

2. Feature

3. Feature

4. Feature

Objekt erkennen

Ergebnis:



Ergebnis der Vorhersage:

Vorhersage: Objekt (0)

Warum hat sich das neuronale Netzwerk so entschieden?

Sicherheit: 99.2 % · Stärkster lokaler Einfluss: Feature 2

Feature 1		→ Objekt 0
Feature 2 ★		→ Objekt 0
Feature 3		→ Objekt 1
Feature 4		→ Objekt 0



To do this, the selected **features** describing the objects, such as their physical characteristics, dimensions, or colour, **are entered into the input form** again. The neural network then determines the **corresponding activation path** (see Section 2), the **strength of each feature's influence**, and the corresponding **confidence of the prediction**. In the AI-ANNE Computer Vision version, a **camera is used** instead, allowing students to directly test the recognition of trained **geometric shapes** and the accuracy of their recognition using the same principle.

Training Language Models with a Generative Transformer

(1) Presets

Embedding Dimensions

The **embedding dimensions** describe **how many numbers** the model uses to represent the **characteristics of a word**. The Transformer processes these embeddings **based on their context** and improves them during training.

Embedding-Dimensionen



(2) Data and (3) Training

AI-ANNE **automatically suggests suitable settings** for training language models with a Generative Transformer. To **better understand how it works**, students are encouraged to experiment with these settings themselves and observe how they affect the learning behaviour of the neural network.

Vocabulary Size

The vocabulary size describes **how many different words or parts of words a language model knows**. You can think of it like a dictionary: The larger the dictionary, the more words the model can understand and use.

Beispielsätze als Trainingsdaten

Die Königin sieht den Hund.
Die Königin ruft den Hund.
Die Königin füttert den Hund.
Die Königin lobt den Hund.
Der Hund sieht die Königin.
Der Hund hört die Königin.
Der Hund folgt der Königin.
Der Hund bewacht das Schloss.
Der Hund sucht den Knochen.

Tokenisieren

Beispieldaten

Vorgeschlagene Trainings-Hyperparameter

Vokabulargröße
28

Batches
111

Kontextlänge
9

Attention-Heads
2



Batches

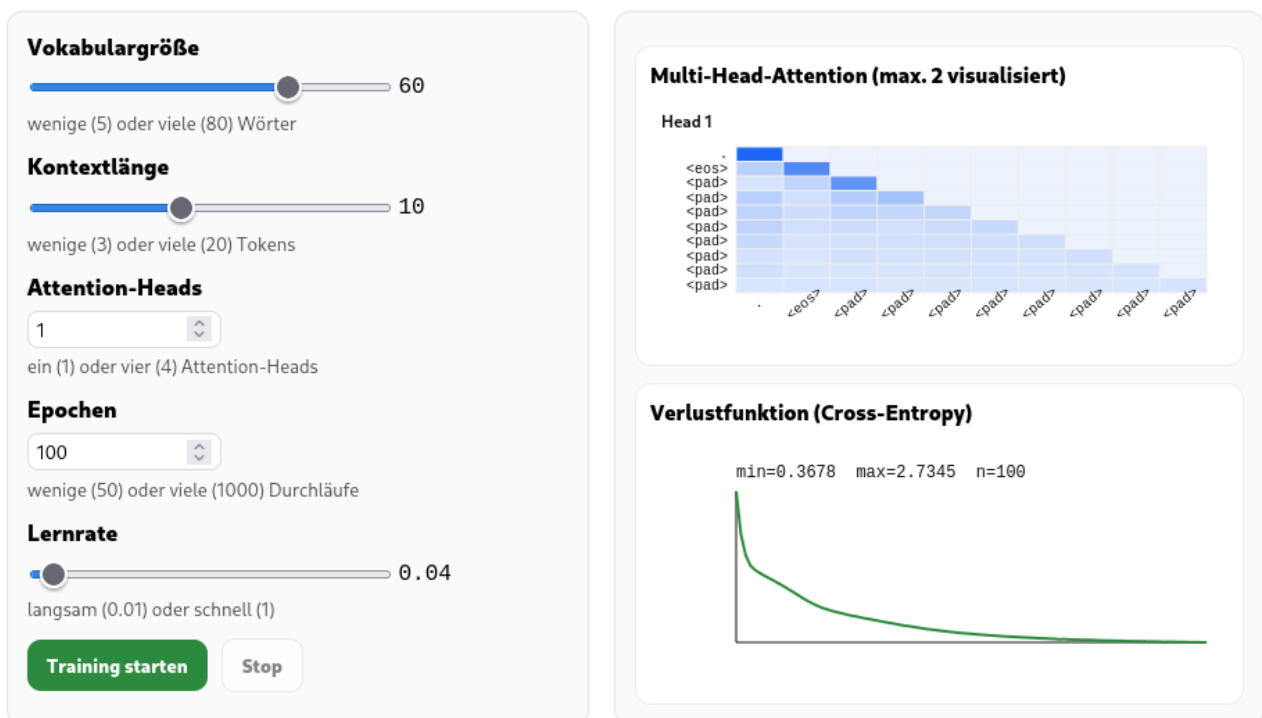
A batch is a **small group of training examples** that the neural network **processes at the same time**. Instead of looking at all example sentences as training data at once, the **neural network learns step by step** from several small groups. After each batch, it adjusts its settings so that it can make better predictions with the next batch.

Context Length

The context length describes **how many words the model can remember and consider at the same time** when reading a text or writing new text. If the neural network with a Transformer can remember many words from the preceding sentence, it can understand the text better.

Attention Heads

Attention heads are different **perspectives of a neural network**. When processing an input, each attention head focuses on different **important relationships**, such as the **meaning of individual words or how they relate to one another**. This allows the neural network to consider information from different perspectives and better understand the relationships between words.



Multi-Head Attention (max. 2 visualized) as an Attention Matrix

A **visualized attention matrix** shows **which words a language model pays particular attention to when reading a sentence**, making it possible to see which words are important for understanding other words.

Loss Function (Cross Entropy)

The **loss function** is a **number that measures how many errors the model is still making**, helping the neural network with its Transformer determine how much it needs to adjust its settings in order to improve during the next learning step.



(4) Embeddings

Vektorized Words (Token Vector)

A neural network cannot understand words directly. Instead, it first converts each **word or token into a list of numbers**. This list of numbers is called a **token vector**. Similar words often have similar token vectors, allowing the model to recognize **relationships between words**. During training, these numbers are gradually adjusted so that the vectors increasingly reflect the meaning and usage of the words. The **embedding dimensions** determine how many numbers are used to represent a token vector.

Token-Vektoren-Matrix

Token	Vektor
.	[-1.810, 0.255, -0.481, 2.064, 0.147, -1.507, ...]
hund	[1.361, 0.449, 0.075, 0.174, 0.549, 0.996, ...]
der	[-0.427, 0.540, 1.144, -0.183, -1.797, 0.724, ...]
königin	[1.262, -0.083, -0.919, -0.923, 0.612, 0.043, ...]
die	[-0.848, -0.307, 0.428, -0.077, -1.203, -0.861, ...]
den	[0.570, 0.254, 1.650, 0.800, -0.853, -0.091, ...]
gärtner	[0.145, 1.566, -1.004, -0.544, -0.227, -0.024, ...]
im	[-1.073, 0.314, -0.785, 0.795, 0.034, 1.228, ...]
knochen	[0.951, -0.498, -0.241, 0.907, 1.171, -0.648, ...]
sieht	[-0.445, -0.328, -0.788, -0.139, 0.183, -0.205, ...]
das	[1.451, 0.095, -0.447, 0.391, 0.047, 0.680, ...]

Token Vector Matrix (Embedding Matrix)

The **token vector matrix** is a large table in which a **token vector is stored for each known token**. Whenever the neural network **processes a token**, it retrieves the corresponding vector from this matrix. During training, the values in the entire token vector matrix are **continuously adjusted**. As a result, the model gradually learns better representations of words and can more accurately capture their **meaning and relationships with other words**.

(5) Sentence Generator

Maximale Sentence Length

The **maximum sentence length** determines **how many tokens may be used to generate a sentence**. Without a fixed limit, a language model might otherwise continue processing tokens indefinitely.

Temperature

The **temperature** determines **how randomly the neural network selects the next token**. At a **low temperature**, the model usually chooses the most likely token, resulting in more **predictable and consistent texts**. At a **higher temperature**, it more frequently considers less likely tokens as well. This makes the responses **more creative and varied**.



Top-k

Top-k limits the selection of possible next tokens. The neural network therefore considers only this **limited selection** and chooses the next token from it. A **small Top-k value leads to more reliable and predictable responses**, while a larger value allows for greater variety and more creative phrasing.

Top-p (Nucleus Sampling)

Top-p limits the selection of possible next tokens to the **most likely tokens whose combined probability reaches a certain threshold**. The neural network therefore considers only as many tokens as necessary to reach this probability threshold. As a result, the size of the selection automatically adapts to the situation, often creating a good balance between **reliable and varied responses**.

Maximale Satzlänge

Sicherheitsgrenze (falls Satzende fehlt)

Temperatur

niedrige (0.2) oder hohe (2.0) Kreativität

Top-k (Anzahl)

wenige (1) oder viele (10) Alternativen

Top-p (Wahrscheinlichkeit)

geringe (0.1) oder hohe (1.0) Wahrscheinlichkeit

Satz generieren

Ergebnis

Generierter Satz:
die königin sitzt im garten.

Sampling-Parameter:
top_k=5, top_p=0.90, Temperatur=0.8

Schrittweise Wortwahl mit Generierungszustand und Alternativen:

1. Token: sitzt
Alternativen:
gibt (0.240)
füttert (0.208)
sitzt (0.192)
lobt (0.189)
sieht (0.171)

2. Token: im
Alternativen:
im (0.810)
königin (0.102)
der (0.035)
den (0.028)
die (0.025)

3. Token: garten
Alternativen:
garten (0.689)
schloss (0.311)

4. Token: .
Alternativen:
. (1.000)

